

Problem Solving Using Python (CSE1012)

Dr. Abdul Kayom, Assistant Professor, SENSE

Agenda / Contents

01 Tuple

02

03

04

05

Tuple

- Like lists, Tuple is another data structure supported by Python
- It is similar to lists but differs in two things:
 - Tuple is a sequence of immutable objects. You can't change values in tuple
 - Tuple are created using parentheses

Creating a Tuple

- Elements are put comma-separated within parentheses
- Example,

`Tup1 = (val1, val2, ...)`

Creating a Tuple

#Creating Tuples of different types of elements

Tup1 = () #creates an empty tuple

Tup2 = (1, 2, 3, 4, 5)

Tup3 = ('a', 'b', 'c', 'd')

Tup4 = ("abc", "def", "ghi")

Tup5 = (1, 'a', "abc", 2.3)

Creating a Tuple

- Any set of multiple, comma-separated values written without an identifying symbol like [], or () are by default treated as Tuple

- Example,

```
a = 1,2,3
```

```
print(type(a))
```

```
<class 'tuple'>
```

Creating a Tuple

- If you want to put only one element in a Tuple, still you have to put comma after the element

```
tup = (10,)
```

```
type(tup)    #<class 'tuple'>
```

```
tup = (10)
```

```
type(tup)    #<class 'int'>
```

Accessing values in a Tuple

- Indices in tuple also starts at 0
- Operations like slice, concatenation can also be performed on a tuple

Accessing values in a Tuple

#Slice operations in a tuple

```
Tup = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

```
Tup1 = Tup[3:6]
```

```
print(Tup1)
```

```
(4, 5, 6)
```

Accessing values in a Tuple

#Slice operations in a tuple

```
Tup = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

```
Tup2 = Tup[:4]
```

```
print(Tup2)
```

```
(1, 2, 3, 4)
```

Accessing values in a Tuple

#Slice operations in a tuple

```
Tup = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

```
Tup3 = Tup[4:]
```

```
print(Tup3)
```

```
(5, 6, 7, 8, 9, 10)
```

Accessing values in a Tuple

- Tuple is immutable
- Values in a tuple can't be changed
- Two tuples can be added to get a third tuple

```
Tup1 = (1, 2, 3)
```

```
Tup2 = (4, 5)
```

```
Tup3 = Tup1 + Tup2
```

```
Print(Tup3)
```

Deleting a Tuple

- Values in a tuple can't be deleted
- However, you can always delete the entire tuple

```
Tup = (1, 2, 3, 4, 5)
```

```
del Tup[0]
```

Traceback (most recent call last):

```
File "<pyshell#19>", line 1, in <module>
```

```
del Tup[0]
```

TypeError: 'tuple' object doesn't support item deletion

Deleting a Tuple

- Values in a tuple can't be deleted
- However, you can always delete the entire tuple

```
Tup = (1, 2, 3, 4, 5)
```

```
del Tup
```

```
print(Tup)
```

Traceback (most recent call last):

```
File "<pyshell#22>", line 1, in <module>
```

```
    print(Tup)
```

NameError: name 'Tup' is not defined

Basic Operation on Tuples

- Like strings and lists, you can perform operations like concatenation, repetition, etc.
- Only thing is that you can't delete or change elements in a tuple

Basic Operation on Tuples

Operation	Expression	Output
Length	<code>len((1,2,3,4,5,6))</code>	6
Concatenation	<code>(1,2,3) + (4,5,6)</code>	(1, 2, 3, 4, 5, 6)
Repetition	<code>('Good.. ')*3</code>	'Good..Good..Good.. '
Membership	<code>5 in (1,2,3,4,5,6,7,8,9)</code>	True
Iteration	<code>for i in (1,2,3,4,5,6,7,8,9,10): print(i,end=' ')</code>	1,2,3,4,5,6,7,8,9,10
Comparison (Use >, <, ==)	<code>Tup1 = (1,2,3,4,5) Tup2 = (1,2,3,4,5) print(Tup1>Tup2)</code>	False
Maximum	<code>max(1,0,3,8,2,9)</code>	9
Minimum	<code>min(1,0,3,8,2,9)</code>	0
Convert to tuple (converts a sequence into a tuple)	<code>tuple("Hello") tuple([1,2,3,4,5])</code>	('H', 'e', 'l', 'l', 'o') (1, 2, 3, 4, 5)

Nested Tuple

- A tuple inside another tuple is called a nested tuple

#Example of nested tuple

```
toppers =(("Arnav", "Bsc", 92.0), ("Chirag", "BCA", 99.0),  
          ("Dhruv", "B.tech", 97))
```

```
for i in toppers:
```

```
    print(i)
```

Nested Tuple

- Even a list can be specified inside a tuple
- #Example of a tuple containing a list inside
- ```
topper=("John", [97, 95, 96, 99])
print(topper)
('John', [97, 96, 99, 95])
```

# Checking index of a Tuple

- Index of an element in a tuple can be searched using index() method

```
Tup = (1, 2, 3, 4)
```

```
Tup.index(3)
```

```
2
```

# Counting elements in a Tuple

- Count of an element in a tuple can be done using count() method

```
Tup = (1, 2, 3, 4, 5, 2, 4, 5)
```

```
Tup.count(2)
```

```
2
```

# Sorting elements in a Tuple

```
Tup = (5, 2, 3, 0, 7, 9)
```

```
Tup1 = sorted(Tup)
```

```
(0, 2, 3, 5, 7, 9)
```

# Tuple Assignment

- Tuple assignment is a very powerful feature in Python
- It allows a tuple of variables on the left side of the assignment operator to be assigned values from a tuple given on the right side of the operator
- Each value is assigned to its respective variable

# Tuple Assignment

```
(val1, val2, val3) = (1, 2, 3)
print(val1, val2, val3)
```

```
Tup = (100, 200, 300)
(val1, val2, val3) = Tup
print(val1, val2, val3)
```

# Tuple Assignment

```
(val1, val2, val3) = (1+2, 2/3, 9%6)
```

```
print(val1, val2, val3)
```

```
3 0.66 3
```



# Swap two values by Tuple assignment

```
a = 2
```

```
b = 3
```

```
print(a, b)
```

```
2 3
```

```
(a, b) = (b, a) #Tuple assignment
```

```
print(a, b)
```

```
3 2
```

# zip() Function

- **The zip() is a built-in function that takes two or more sequences and "zips" them into a list of tuples.**
- **The tuple thus, formed has one element from each sequence.**

# zip() Function

Example: Program to show the use of zip() function

```
Tup = (1,2,3,4,5)
List1 = ['a','b','c','d','e']
print(list(zip(Tup, List1)))
```

## **OUTPUT**

```
[(1, 'a'), (2, 'b'), (3, 'c'), (4, 'd'), (5, 'e')]
```

# Advantages of Tuple

- **Tuples are used to store values of different data types.**
- **Lists can however, store data of similar data types.**

# Advantages of Tuple

- **Since tuples are immutable, iterating through tuples is faster than iterating over a list.**

# Advantages of Tuple

- **Tuples can be used as key for a dictionary but lists cannot be used as keys.**
- **Tuples are best suited for storing data that is write-protected.**

# Advantages of Tuple

- **Tuples can be used in place of lists where the number of values is known and small.**
- **If you are passing a tuple as an argument to a function, then the potential for unexpected behavior due to aliasing gets reduced.**

# Advantages of Tuple

- **Multiple values from a function can be returned using a tuple.**
- **Tuples are used to format strings.**



# Thank You

**Dr. Abdul Kayom**



**Asst. Prof, (SENSE)**



**kayomabdul@vitap.ac.in**



**8474860025**